Apache Kafka — Guida alle Configurazioni di Producer e Consumer (NSDS)

1. Matrice Rapida: Quali Configurazioni Impostare su Producer e Consumer

Scenario / Garanzia Richiesta	Configurazioni `ProducerConfig` (`Properties`)	Configurazioni `ConsumerConfig` (`Properties`)	Metodi Chiave nel Codice
1. Base (Default / Fire-and-Forget + Auto-Commit)	`BOOTSTRAP_SERVERS_CONFIG = "localhost:9092"` `KEY_SERIALIZER_CLASS_CONFIG = ...` `VALUE_SERIALIZER_CLASS_CONFIG = ...`	`BOOTSTRAP_SERVERS_CONFIG = "localhost:9092"` `GROUP_ID_CONFIG = "groupA"` `ENABLE_AUTO_COMMIT_CONFIG = "true"` `AUTO_COMMIT_INTERVAL_MS_CONFIG = "15000"` `AUTO_OFFSET_RESET_CONFIG = "latest"` `KEY_DESERIALIZER_CLASS_CONFIG = ...` `VALUE_DESERIALIZER_CLASS_CONFIG = ...`	**Producer**: `producer.send(record);` (asincrono) **Consumer**: `consumer.poll(...)` (gli offset sono salvati in automatico ogni 15s)
2. Producer Sincrono (Attesa ACK dal Broker)	Stesse configurazioni del Producer Base	Qualsiasi Consumer	`Future<RecordMetadata> f = producer.send(record);` `RecordMetadata ack = f.get();` (legge `topic()` , `partition()` , `offset()`)
3. Producer Idempotente (No duplicati da retry tra Producer e Partizione)	Base + `ENABLE_IDEMPOTENCE_CONFIG = "true"`	Qualsiasi Consumer	Il producer numera i messaggi (*sequence numbers*); il broker scarta automaticamente i duplicati dovuti a ritrasmissioni
4. Consumer con Commit Manuale (*At-Least-Once*)	Qualsiasi Producer	Base ma con: `ENABLE_AUTO_COMMIT_CONFIG = "false"`	Chiamare `consumer.commitSync()` **dopo** aver elaborato il batch o ogni *N* messaggi (`commitEvery`)
5. Scrittura e Lettura Transazionale (Tutto-o-Niente)	Base + `TRANSACTIONAL_ID_CONFIG = "myTxId"` *(implica `ENABLE_IDEMPOTENCE_CONFIG = "true"`)*	Base + `ISOLATION_LEVEL_CONFIG = "read_committed"` *(scarta i messaggi delle transazioni abortite)*	**Producer**: `initTransactions()` (1 volta) → `beginTransaction()` → `send(...)` → `commitTransaction()` (o `flush()` + `abortTransaction()`)
6. Atomic Forwarder Stateless (*End-to-End Exactly-Once* Read–Process–Write — Lab 1 Es. 3)	Base + `TRANSACTIONAL_ID_CONFIG = "txId"` *(univoco per istanza!)* `ENABLE_IDEMPOTENCE_CONFIG = "true"`	Base + `ENABLE_AUTO_COMMIT_CONFIG = "false"` `ISOLATION_LEVEL_CONFIG = "read_committed"`	Il consumer **non** usa `commitSync()` . Il producer committa sia i messaggi prodotti sia gli offset del consumer (`lastOffset + 1`) con: `producer.sendOffsetsToTransaction(map, consumer.groupMetadata());` `producer.commitTransaction();` In caso di `KafkaException` : `abortTransaction()` + `consumer.seek(...)` .
7. Atomic Forwarder *Stateful* Fault-Tolerant (EOS con stato locale, es. Contatore per chiave — Lab 1 Es. 4)	Come Atomic Forwarder + `VALUE_SERIALIZER_CLASS_CONFIG = IntegerSerializer...` (se l'output è il conteggio)	Consumer principale come sopra + **`storeConsumer`** all'avvio (`ISOLATION_LEVEL_CONFIG = "read_committed"`) per rileggere `outputTopic`	**All'avvio (prima di `subscribe(inputTopic)`)**: `storeConsumer.partitionsFor(outputTopic)` → `assign(partitions)` → `seekToBeginning(partitions)` → `poll()` per ricostruire la `Map<K, Integer>` in RAM. **Topic di output**: creato con **Log Compaction** (`CLEANUP_POLICY_COMPACT`).
8. Gestione Topic e Log Compaction (`AdminClient` / `TopicManager`)	`AdminClientConfig.BOOTSTRAP_SERVERS_CONFIG = "localhost:9092"`	Configurazioni `NewTopic` (opzionali per topic compattato): `CLEANUP_POLICY_CONFIG = CLEANUP_POLICY_COMPACT` `MIN_CLEANABLE_DIRTY_RATIO_CONFIG = "0.01"` `SEGMENT_MS_CONFIG = "5000"`	`AdminClient.create(props)` → `listTopics()` → `deleteTopics(...)` → `new NewTopic(name, partitions, replFactor).configs(map)` → `createTopics(...)`

2. Configurazioni del PRODUCER (`ProducerConfig`) — Analisi Dato per Dato

2.1 Tabella di Tutte le Proprietà del Producer

Costante `ProducerConfig.*`	Chiave Stringa	Valore da Passare (`String`)	Default	Effetto sul Comportamento del Producer
`BOOTSTRAP_SERVERS_CONFIG`	`"bootstrap.servers"`	`"localhost:9092"`	*Obbligatorio*	Indirizzo iniziale di uno o più broker Kafka per connettersi al cluster e scaricare i metadati delle partizioni.
`KEY_SERIALIZER_CLASS_CONFIG`	`"key.serializer"`	`StringSerializer.class.getName()`	*Obbligatorio*	Specifica come convertire la chiave `K` del `ProducerRecord<K, V>` in `byte[]` .
`VALUE_SERIALIZER_CLASS_CONFIG`	`"value.serializer"`	`StringSerializer.class.getName()`	*Obbligatorio*	Specifica come convertire il valore `V` del `ProducerRecord<K, V>` in `byte[]` .
`ENABLE_IDEMPOTENCE_CONFIG`	`"enable.idempotence"`	`String.valueOf(true)` oppure `String.valueOf(false)`	`"false"`	**Se `"true"`** : garantisce **Exactly-Once tra il producer e la partizione**. Evita che un messaggio venga scritto due volte se l'ACK si perde e il producer lo ritrasmette.
`TRANSACTIONAL_ID_CONFIG`	`"transactional.id"`	`"myTransactionalId"` *(Stringa **univoca per ogni istanza** di producer)*	`null`	**Abilita le transazioni**. Registra il producer presso il *Transaction Coordinator* del broker per eseguire scritture atomiche su più partizioni/topic e isolare istanze duplicate (*zombie fencing*).
`ACKS_CONFIG` *(opzionale)*	`"acks"`	`"0"` , `"1"` , `"all"`	`"1"` (`"all"` se idempotente)	Numero di repliche che devono confermare la scrittura: `"0"` (nessun ACK), `"1"` (solo la replica leader), `"all"` (leader + tutte le repliche follower in-sync).

2.2 Approfondimento: Cosa Cambia Togliendo o Aggiungendo le Configurazioni del Producer

A) Chiave del Messaggio (`key`) e Scelta della Partizione in `ProducerRecord<K, V>`

Il producer decide su quale partizione del topic scrivere il messaggio in base a come costruisci `ProducerRecord` :

- `new ProducerRecord<>(topic, key, value)` *(con chiave specificata)*:
- Il partitioner di default calcola un **hash della chiave** (`hash(key) % numPartitions`).
- **Garanzia fondamentale**: tutti i messaggi con la **stessa** `key` finiscono **sempre nella stessa partizione** e vengono quindi salvati e letti in **ordine FIFO rigoroso**.
- **Implicazione per i Consumer Stateful (Lab 1 Es. 4)**: poiché ogni partizione è assegnata a un solo consumer di un Consumer Group, usare la `key` garantisce che tutti i messaggi con la stessa chiave vengano sempre elaborati dalla **stessa istanza di consumer**!
- `new ProducerRecord<>(topic, value)` *(senza chiave, cioè `key = null`)*:
- I messaggi vengono distribuiti tra le partizioni disponibili per bilanciare il carico, ma **non c'è alcuna garanzia di località** né di ordine relativo tra messaggi diversi.
- `new ProducerRecord<>(topic, partitionIndex, key, value)` *(con partizione forzata)*:
- Scavalca l'hash della chiave e scrive direttamente nella partizione `partitionIndex` (`0, 1, ...`).

B) Invio Asincrono vs. Invio Sincrono (`waitAck`)

`producer.send(record)` è **sempre asincrono** di base e restituisce immediatamente un `Future<RecordMetadata>` :

- `waitAck = false` **(Asincrono)**: chiami solo `producer.send(record);` e prosegui subito col messaggio successivo (massimo throughput).
- `waitAck = true` **(Sincrono / Bloccante)**: chiami `RecordMetadata ack = producer.send(record).get();` . Il thread del producer si blocca finché il broker non conferma la scrittura e ti restituisce i dati in `ack` :
- `ack.topic()` → nome del topic (`String`)
- `ack.partition()` → numero della partizione in cui è stato scritto (`int`)
- `ack.offset()` → posizione sequenziale assegnata nel log della partizione (`long`)

C) `ENABLE_IDEMPOTENCE_CONFIG = "true"` (Producer Idempotente)

- **Problema senza idempotenza (`"false"`)**: il producer invia un messaggio `M1` al broker; il broker lo scrive nella partizione ma l'ACK si perde sulla rete. Scatta il timeout e il producer rinvia `M1` : il broker lo scrive una seconda volta (*At-Least-Once* → **messaggio duplicato**).
- **Soluzione con** `ENABLE_IDEMPOTENCE_CONFIG = "true"` : il broker assegna al producer un ID univoco (`PID`) e il producer allega un **numero di sequenza crescente** a ogni messaggio inviato verso una partizione. Se il broker riceve di nuovo `M1` con un sequence number già visto, **scarta il duplicato** e risponde semplicemente con l'ACK (*Exactly-Once* sulla singola partizione).

D) `TRANSACTIONAL_ID_CONFIG` (Producer Transazionale: `commit` , `abort` , `flush` e *Zombie Fencing* — Lab 1 Es. 3)

- **Quando serve**: quando vuoi scrivere **più messaggi** (anche su topic e partizioni diverse) **in modo atomico** (*o vengono scritti tutti, o nessuno*), oppure quando vuoi fare un *Atomic Forwarder* (scrivere l'output e committare gli offset del consumer nella stessa transazione).

- **Regole obbligatorie nel codice**:
- Impostare `props.put(ProducerConfig.TRANSACTIONAL_ID_CONFIG, "myTransactionalId");` (e `ENABLE_IDEMPOTENCE_CONFIG = "true"`, che viene comunque attivato di default dalle transazioni).
- Chiamare `producer.initTransactions();` **una sola volta** prima del ciclo `for` / `while`.
- Per ogni transazione:
  - Aprire con `producer.beginTransaction();`
  - Inviare uno o più record con `producer.send(record);`
  - Chiudere con `producer.commitTransaction();` (conferma) oppure `producer.abortTransaction();` (annulla).
- **Perché in `TransactionalProducer.java` si chiama `producer.flush()` prima di `producer.abortTransaction()`?**:
- `producer.send(record)` mette i messaggi in un buffer locale in RAM del producer.
- Se chiami subito `abortTransaction()` senza `flush()`, i messaggi vengono semplicemente cancellati dal buffer locale del producer senza mai arrivare al broker.
- Chiamando `producer.flush()` prima di `abortTransaction()`, forzi l'invio fisico dei messaggi sul disco del broker, seguiti subito dal marker di transazione abortita. Così puoi vedere che un consumer `"read_uncommitted"` li legge comunque, mentre un consumer `"read_committed"` li scarta!
- **Attenzione: Cosa succede se avvii PIÙ istanze di un *Atomic Forwarder* (Lab 1 Es. 3)?**:
- Per scalare in parallelo, tutte le istanze devono condividere lo **stesso** `group.id` del Consumer, ma **ogni istanza DEVE avere un** `transactional.id` **DIVERSO** (es. passato da riga di comando `args[0]`: `"forwarder-1"`, `"forwarder-2"`).
- **Zombie Fencing**: se avvii due istanze contemporaneamente con lo **stesso** `transactional.id`, quando la seconda chiama `producer.initTransactions()`, il *Transaction Coordinator* incrementa l'*epoch* associata a quel `transactional.id`, annulla eventuali transazioni aperte della prima istanza e **taglia fuori ("fences off") la prima istanza**, che alla successiva operazione transazionale fallisce lanciando `ProducerFencedException`!
- **Gestione delle Eccezioni in un ciclo transazionale (`try-catch` di `AtomicUpperCaseConsumer.java` — Lab 1 Es. 3)**:
- `ProducerFencedException` | `OutOfOrderSequenceException`: errori fatali non recuperabili (un'altra istanza con lo stesso `transactional.id` ha preso il sopravvento o la sequenza è corrotta) → chiudere `producer.close()`, `consumer.close()` e uscire dal ciclo (`break`).
- `KafkaException`: errore recuperabile durante la transazione → chiamare `producer.abortTransaction()` e **riportare indietro gli offset del consumer** tramite `consumer.seek(partition, firstOffset)` per ogni partizione del batch (verificando `consumer.assignment().contains(partition)`), così che il prossimo `poll()` rilegga i record della transazione abortita invece di perderli!

---

3. Configurazioni del CONSUMER (`ConsumerConfig`) — Analisi Dato per Dato

3.1 Tabella di Tutte le Proprietà del Consumer

Costante `ConsumerConfig.*`	Chiave Stringa	Valore da Passare (`String`)	Default	Effetto sul Comportamento del Consumer
`BOOTSTRAP_SERVERS_CONFIG`	`"bootstrap.servers"`	`"localhost:9092"`	*Obbligatorio*	Indirizzo iniziale del/dei broker Kafka a cui connettersi.
`GROUP_ID_CONFIG`	`"group.id"`	`"groupA"`, `"groupB"`, ecc.	*Obbligatorio*	Identifica il **Consumer Group** a cui appartiene il consumer. Controlla la ripartizione delle partizioni e il salvataggio degli offset.
`KEY_DESERIALIZER_CLASS_CONFIG`	`"key.deserializer"`	`StringDeserializer.class.getName()`	*Obbligatorio*	Converte i `byte[]` della chiave letta dal broker nel tipo Java `K` (deve combaciare col `KEY_SERIALIZER` del producer).
`VALUE_DESERIALIZER_CLASS_CONFIG`	`"value.deserializer"`	`StringDeserializer.class.getName()`	*Obbligatorio*	Converte i `byte[]` del valore letto dal broker nel tipo Java `V` (deve combaciare col `VALUE_SERIALIZER` del producer).
`ENABLE_AUTO_COMMIT_CONFIG`	`"enable.auto.commit"`	`String.valueOf(true)` oppure `String.valueOf(false)`	`"true"`	**Se `"true"`**: il consumer salva gli offset da solo a intervalli regolari. **Se `"false"`**: il salvataggio automatico è spento; devi committare tu con `consumer.commitSync()` o tramite transazione del producer.
`AUTO_COMMIT_INTERVAL_MS_CONFIG`	`"auto.commit.interval.ms"`	`"15000"` (15s), `"5000"` (5s)	`"5000"`	Ogni quanti millisecondi il consumer salva in background l'offset

Costante `ConsumerConfig.*`	Chiave Stringa	Valore da Passare (`String`)	Default	Effetto sul Comportamento del Consumer
				dell'ultimo `poll()` (attivo solo se `enable.auto.commit = "true"`).
`AUTO_OFFSET_RESET_CONFIG`	`"auto.offset.reset"`	`"latest"` oppure `"earliest"`	`"latest"`	Stabilisce da dove partire **SOLO SE non esiste alcun offset già salvato** per quel `group.id` : - `"latest"` : legge solo i **nuovi** messaggi prodotti dopo l'avvio. - `"earliest"` : legge **tutti** i messaggi fin dall'offset `0` .
`ISOLATION_LEVEL_CONFIG`	`"isolation.level"`	`"read_committed"` oppure `"read_uncommitted"`	`"read_uncommitted"`	Stabilisce cosa leggere se il producer usa transazioni: - `"read_uncommitted"` (default): legge tutto, **anche i messaggi abortiti**. - `"read_committed"` : legge **solo** i messaggi di transazioni committate (e quelli non transazionali).

3.2 Approfondimento: Come Funzionano le Configurazioni del Consumer

A) `GROUP_ID_CONFIG` : Stesso Gruppo (Load Balancing) vs. Gruppi Diversi (Broadcast)

Ogni Consumer Group rappresenta **un singolo consumer logico**. Dato un topic con ***P* partizioni**:

Configurazione di `GROUP_ID_CONFIG`	Come vengono assegnate le *P* partizioni?	Chi riceve i messaggi del topic?
1 solo Consumer con `group.id = "groupA"`	Tutte le *P* partizioni sono assegnate a quell'unico consumer.	Riceve tutti i messaggi di tutte le partizioni.
***C* ≤ *P* Consumer** con lo **STESSO** `group.id = "groupA"`	Le *P* partizioni vengono **divise (partizionate)** tra i *C* consumer (una partizione è assegnata a **un solo** consumer del gruppo).	**Load Balancing**: ogni messaggio viene letto da **uno solo** dei consumer del gruppo (elaborazione parallela).
***C* > *P* Consumer** con lo **STESSO** `group.id = "groupA"`	*P* consumer ricevono 1 partizione ciascuno; i restanti ***C* - *P* consumer restano fermi (inattivi / idle)**.	**Limite di scalabilità**: non ha mai senso avere più consumer nello stesso gruppo del numero di partizioni *P* del topic!
Più Consumer con **DIVERSO** `group.id` (es. `"groupA"` e `"groupB"`)	Ogni gruppo riceve tutte le *P* partizioni in modo indipendente e mantiene il proprio offset separato.	**Broadcast (Pub/Sub)**: ogni messaggio viene consegnato a **ciascun** Consumer Group iscritto al topic.

B) `AUTO_OFFSET_RESET_CONFIG` (`"latest"` vs `"earliest"`) e Interazione con `GROUP_ID_CONFIG`

Attenzione (errore classico d'esame/lab): `AUTO_OFFSET_RESET_CONFIG` entra in gioco **esclusivamente quando per il** `GROUP_ID_CONFIG` **scelto non esiste ancora nessun offset salvato** nel topic `__consumer_offsets` di Kafka!

Stato del `GROUP_ID_CONFIG` sul Broker	Valore di `AUTO_OFFSET_RESET_CONFIG`	Cosa fa il Consumer all'avvio?
Gruppo nuovo (primo avvio assoluto di `"groupA"`)	`"latest"` *(default)*	Si posiziona alla **fine** del log: ignora tutti i messaggi già presenti e aspetta solo quelli nuovi inviati da questo momento in poi.
Gruppo nuovo (primo avvio assoluto di `"groupA"`)	`"earliest"`	Si posiziona all'**inizio** del log (`offset = 0`): scarica e legge tutta la storia dei messaggi già presenti nel topic.
Gruppo già esistente (`"groupA"` ha già fatto commit in passato)	`"latest"` **oppure** `"earliest"`	**Ignora** `AUTO_OFFSET_RESET_CONFIG` **!** Riparte esattamente dall'**ultimo offset committato** da `"groupA"` prima di spegnersi. *(Se vuoi rileggere tutto da zero con `"earliest"` , devi cambiare il nome di `GROUP_ID_CONFIG` , es. `"groupB"` !)*

C) `ENABLE_AUTO_COMMIT_CONFIG` : I 3 Modi di Gestire il Commit degli Offset

1. **Modo 1 — Auto-Commit Automatico (`ENABLE_AUTO_COMMIT_CONFIG = "true"` , default)**:
   - Imposti anche `AUTO_COMMIT_INTERVAL_MS_CONFIG = "15000"` (15 secondi).
   - Il consumer salva automaticamente gli offset ogni 15s durante le chiamate a `poll()` .

- *Problema*: se il consumer crasha a 14 secondi dopo aver già stampato/elaborato dei messaggi ma prima dell'auto-commit, al riavvio **rileggerà gli stessi messaggi** (duplicati). Viceversa, se l'auto-commit scatta appena ricevuti i record da `poll()` ma il consumer crasha a metà del ciclo `for` prima di finirne l'elaborazione, quei messaggi **non verranno più riletti** (perdita).

2. **Modo 2 — Commit Manuale dal Consumer (`ENABLE_AUTO_COMMIT_CONFIG = "false"` + `consumer.commitSync()`)**:
   - Il consumer non salva mai gli offset da solo.
   - Nel ciclo di lettura, **dopo** aver elaborato il messaggio (o ogni `commitEvery = 100` messaggi, oppure alla fine del ciclo `for` di `poll()`), chiami esplicitamente `consumer.commitSync();`.
   - Garantisce **At-Least-Once**: nessun messaggio viene mai committato prima di essere stato elaborato.
3. **Modo 3 — Commit Transazionale tramite il Producer (`ENABLE_AUTO_COMMIT_CONFIG = "false"` + `producer.sendOffsetsToTransaction(...)`)**:
   - Usato nell'**Atomic Forwarder** (quando un componente è sia Consumer di `inputTopic` sia Producer verso `outputTopic`).
   - Il consumer ha `ENABLE_AUTO_COMMIT_CONFIG = "false"`, ma **non chiama mai** `consumer.commitSync()`.
   - È il **producer** che, dentro la sua transazione, calcola per ogni `TopicPartition` l'ultimo offset letto (`lastOffset`) e invia `new OffsetAndMetadata(lastOffset + 1)` al *Transaction Coordinator* tramite `producer.sendOffsetsToTransaction(map, consumer.groupMetadata());` seguito da `producer.commitTransaction();`.
   - **Perché `lastOffset + 1` ?**: perché l'offset salvato in Kafka indica sempre **la prossima casella da leggere**. Se l'ultimo messaggio letto ha offset `9`, devi salvare `10` affinché la prossima lettura parta dal messaggio `10`!

D) `ISOLATION_LEVEL_CONFIG`: `"read_committed"` vs. `"read_uncommitted"`

Quando un `TransactionalProducer` scrive su un topic, i suoi messaggi possono essere **committati** (`commitTransaction()`) oppure **abortiti** (`abortTransaction()`).

- `ISOLATION_LEVEL_CONFIG = "read_uncommitted"` *(Attenzione: è il **DEFAULT** di Kafka!)*:
- Il consumer legge **tutti** i messaggi presenti fisicamente nella partizione, **compresi quelli di transazioni ancora aperte o abortite** (se erano stati inviati al broker tramite `flush()`).
- `ISOLATION_LEVEL_CONFIG = "read_committed"`:
- Il consumer filtra e restituisce **solo** i messaggi appartenenti a transazioni che hanno eseguito `commitTransaction()` con successo (oltre ai normali messaggi inviati da producer non transazionali).
- **Va sempre impostato a** `"read_committed"` in tutti gli esercizi che parlano di transazioni, *Atomic Forwarder* o *Exactly-Once Semantics (EOS)*.

E) `subscribe(topics)` vs. `assign(partitions)` + `seek` / `seekToBeginning` (Lab 1 Es. 3 & Es. 4)

Oltre a `consumer.subscribe(...)`, Kafka mette a disposizione metodi per controllare manualmente quali partizioni leggere e da quale offset:

Metodo del Consumer	Come funziona	Quando si usa nei Lab / Esami
`consumer.subscribe(Collections.singletonList(topic))`	Iscrizione dinamica tramite **Consumer Group**: il broker assegna e revoca automaticamente le partizioni (*rebalance*) quando entrano o escono istanze con lo stesso `group.id`.	Lettura standard del flusso di input (`inputTopic`) in quasi tutti gli esercizi.
`consumer.partitionsFor(topic)` + `consumer.assign(partitions)`	Assegnazione **manuale e statica** di una lista esplicita di `TopicPartition`. **Disabilita il rebalance automatico del gruppo**: l'istanza legge esattamente tutte le partizioni passate ad `assign`.	**Lab 1 Es. 4 (`storeConsumer`)**: all'avvio di un consumer *stateful*, per leggere **tutte** le partizioni del topic di stato/output dall'inizio senza subire rebalance.
`consumer.seekToBeginning(partitions)`	Sposta forzatamente il cursore di lettura all'**offset `0`** per le partizioni indicate, **ignorando qualsiasi offset committato in precedenza**.	**Lab 1 Es. 4**: subito dopo `storeConsumer.assign(partitions)`, per rileggere tutta la storia di `outputTopic` e ricostruire la `Map` in memoria dopo un crash.
`consumer.seek(partition, offset)`	Sposta manualmente il cursore di una specifica `TopicPartition` all' `offset` desiderato.	**Lab 1 Es. 3**: nel blocco `catch (KafkaException e)` dopo `producer.abortTransaction()`, per riavvolgere il consumer al primo offset del batch fallito (`records.records(partition).get(0).offset()`) e rielaborarlo al `poll()` successivo.

F) Elaborazione *Stateful* Fault-Tolerant con EOS e *Log Compaction* (Lab 1 Es. 4)

Nell' `AtomicUpperCaseConsumer.java` (Lab 1 Es. 3), la trasformazione è **stateless** (ogni messaggio viene convertito in minuscolo indipendentemente dai precedenti). Cosa succede invece se il forwarder mantiene uno **stato locale in RAM**, ad esempio una `Map<String, Integer> countMap` con il numero totale di messaggi ricevuti per ciascuna `key` (`AtomicFaultTolerantForwarder.java`)?

1. **Perché l'Atomic Forwarder dell'Es. 3 da solo NON garantisce EOS se c'è uno stato in RAM?**:
   - Se il processo crasha dopo aver committato 10 messaggi per `"Key1"` (`count = 10`), al riavvio Kafka fa ripartire il consumer dall'11° messaggio su `inputTopic`, **ma la `HashMap` in RAM riparte vuota (`countMap` è nuova)**!
   - L'11° messaggio verrebbe quindi pubblicato su `outputTopic` con `count = 1` anziché `11`, perdendo lo stato accumulato prima del crash.

2. **Come rendere lo stato Fault-Tolerant garantendo *Exactly-Once* (Singola Istanza)**:
   - Poiché a ogni messaggio letto da `inputTopic` (`topicA`) il forwarder pubblica su `outputTopic` (`topicB`) la coppia `(key, newCount)` **dentro la stessa transazione** che committa gli offset di `topicA` , il topic `topicB` funge automaticamente da **registro di stato persistente (changelog)** perfettamente allineato con gli offset consumati!
   - **Fase di Recovery all'avvio**: prima di chiamare `consumer.subscribe(inputTopic)` , il programma crea un consumer temporaneo `storeConsumer` (con `ISOLATION_LEVEL_CONFIG = "read_committed"` e `IntegerDeserializer`), recupera tutte le partizioni di `outputTopic` con `partitionsFor(outputTopic)` , se le assegna con `assign(partitions)` , si posiziona all'inizio con `seekToBeginning(partitions)` e legge tutti i record già presenti (`poll(Duration.ofSeconds(1))` finché `stored.isEmpty()`), eseguendo `countMap.put(r.key(), r.value())` .
   - Poiché i messaggi in ogni partizione sono ordinati cronologicamente, l'ultimo valore letto per ogni `key` sovrascrive i precedenti nella `HashMap` , ripristinando esattamente il conteggio dell'ultima transazione committata!
3. **Perché attivare la *Log Compaction* su `outputTopic` (`TopicManager.java`)?**:
   - Per ricostruire `countMap` non serve conservare tutti i conteggi intermedi (`Key1 -> 1` , `Key1 -> 2` , ..., `Key1 -> 99`), ma basta **solo l'ultimo valore per ogni chiave** (`Key1 -> 100`).
   - Impostando `TopicConfig.CLEANUP_POLICY_CONFIG = TopicConfig.CLEANUP_POLICY_COMPACT` su `topicB` , Kafka compatta periodicamente il log in background eliminando i record obsoleti con la stessa chiave e conservando solo l'ultimo valore per ciascuna `key` .
4. **Cosa cambia con PIÙ istanze in parallelo (`Multiple Instances` — Domanda chiave Lab 1 Es. 4)?**:
   - **Durante il funzionamento normale (senza crash/rebalance)**: poiché il producer assegna la partizione in base a `hash(key)` , tutti i messaggi con la stessa `key` finiscono sempre nella stessa partizione di `topicA` e vengono quindi consegnati **sempre alla stessa istanza** del Consumer Group. Ogni istanza mantiene correttamente i contatori del proprio sottoinsieme di chiavi.
   - **Il problema in caso di Crash / Rebalance dinamico con più istanze**:
     - Supponiamo che l'istanza C_1 gestisca la `Partition 0` e l'istanza C_2 gestisca la `Partition 1` . Entrambe hanno letto `topicB` solo al momento del loro avvio.
     - Se dopo 5 minuti C_1 crasha, Kafka riassegna la `Partition 0` a C_2 tramite *rebalance*. Tuttavia, C_2 **non ha in RAM gli aggiornamenti** che C_1 ha scritto su `topicB` negli ultimi 5 minuti! Se C_2 iniziasse subito a processare la `Partition 0` , userebbe conteggi vecchi.
     - **Soluzione per più istanze**: oltre a dare a ogni istanza un `transactional.id` distinto, bisogna passare a `consumer.subscribe(topics, new ConsumerRebalanceListener() { ... })` un listener che, ad ogni `onPartitionsAssigned(...)` (cioè ogni volta che avviene un rebalance), **rilegge `outputTopic` dall'inizio** (`assign` + `seekToBeginning`) per aggiornare `countMap` prima che il consumer elabori i nuovi messaggi delle partizioni appena ereditate!

3.3 Corrispondenza tra Tipi `<K, V>` e Classi Serializer / Deserializer

I tipi generici di `KafkaProducer<K, V>` e `KafkaConsumer<K, V>` devono corrispondere alle classi impostate nelle `Properties` (package `org.apache.kafka.common.serialization.*`):

Tipo di Dato Java (`K` o `V`)	`ProducerConfig` (`KEY_SERIALIZER_CLASS_CONFIG` / `VALUE_SERIALIZER_CLASS_CONFIG`)	`ConsumerConfig` (`KEY_DESERIALIZER_CLASS_CONFIG` / `VALUE_DESERIALIZER_CLASS_CONFIG`)
`String`	`StringSerializer.class.getName()`	`StringDeserializer.class.getName()`
`Integer`	`IntegerSerializer.class.getName()`	`IntegerDeserializer.class.getName()`
`Long`	`LongSerializer.class.getName()`	`LongDeserializer.class.getName()`
`Double`	`DoubleSerializer.class.getName()`	`DoubleDeserializer.class.getName()`
`byte[]`	`ByteArraySerializer.class.getName()`	`ByteArrayDeserializer.class.getName()`

---

4. Tutti i Template di Configurazione Pronti all'Uso (Codice Completo)

4.1 Producer Base (Asincrono e Sincrono con `waitAck`) + Producer Idempotente

(Rif: `BasicProducer.java` , `IdempotentProducer.java`)

```
// --- CONFIGURAZIONE PRODUCER (BASE O IDEMPOTENTE) ---
final Properties props = new Properties();
props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());
props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());

// Decommenta la riga seguente per avere un PRODUCER IDEMPOTENTE (Exactly-Once tra producer e partizione):
// props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, String.valueOf(true));

final KafkaProducer<String, String> producer = new KafkaProducer<>(props);

// Creazione e invio del record (stessa key -> stessa partizione garantita)
final ProducerRecord<String, String> record = new ProducerRecord<>("topicA", "Key1", "Val1");
final Future<RecordMetadata> future = producer.send(record);

// Opzionale: se waitAck == true, attende la conferma sincrona dal broker e legge partizione e offset
if (waitAck) {
    try {
        RecordMetadata ack = future.get();
        System.out.println("Ack for topic " + ack.topic()
                + ", partition " + ack.partition()
                + ", offset " + ack.offset());
    } catch (InterruptedException | ExecutionException e) {
        e.printStackTrace();
    }
}

producer.close();
```

4.2 Consumer Base (Auto-Commit) vs. Consumer con Commit Manuale (`commitSync`)

(Rif: `BasicConsumer.java` , `BasicConsumerManual.java`)

```
// --- CONFIGURAZIONE CONSUMER (AUTO-COMMIT VS COMMIT MANUALE) ---
final boolean autoCommit = false;          // true = Auto-Commit, false = Commit Manuale con commitSync()
final int autoCommitIntervalMs = 15000;    // Usato solo se autoCommit == true
final int commitEvery = 100;               // Usato solo se autoCommit == false
final String offsetResetStrategy = "latest"; // "latest" (solo nuovi) oppure "earliest" (da offset 0 se gruppo nuovo)

final Properties props = new Properties();
props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
props.put(ConsumerConfig.GROUP_ID_CONFIG, "groupA");
props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, String.valueOf(autoCommit));
if (autoCommit) {
    props.put(ConsumerConfig.AUTO_COMMIT_INTERVAL_MS_CONFIG, String.valueOf(autoCommitIntervalMs));
}
props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, offsetResetStrategy);
props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());

KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props);
consumer.subscribe(Collections.singletonList("topicA"));

int numConsumed = 0;
while (true) {
    final ConsumerRecords<String, String> records = consumer.poll(Duration.of(5, ChronoUnit.MINUTES));
    for (final ConsumerRecord<String, String> record : records) {
        System.out.println("Partition: " + record.partition()
                + "\tOffset: " + record.offset()
                + "\tKey: " + record.key()
                + "\tValue: " + record.value());

        if (!autoCommit) {
            numConsumed++;
            if (numConsumed == commitEvery) {
                numConsumed = 0;
                consumer.commitSync(); // Salva manualmente gli offset sul broker ogni 100 messaggi
            }
        }
    }
}
```

4.3 Producer Transazionale e Consumer Transazionale (`read_committed` vs `read_uncommitted`)

(Rif: `TransactionalProducer.java` , `TransactionalConsumer.java`)

```
// ========================================================================
// 1. TRANSACTIONAL PRODUCER
// ========================================================================
final Properties producerProps = new Properties();
producerProps.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
producerProps.put(ProducerConfig.TRANSACTIONAL_ID_CONFIG, "myTransactionalId"); // Attiva le transazioni!
producerProps.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());
producerProps.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());

final KafkaProducer<String, String> producer = new KafkaProducer<>(producerProps);
producer.initTransactions(); // OBBLIGATORIO prima di iniziare qualsiasi transazione!

for (int i = 0; i < 100; i++) {
    final ProducerRecord<String, String> record = new ProducerRecord<>("topicA", "Key" + i, "Val" + i);

    producer.beginTransaction();
    producer.send(record);
    if (i % 2 == 0) {
        producer.commitTransaction(); // Conferma la transazione (i pari sono committati)
    } else {
        producer.flush();             // Scrive sul log prima di abortire (altrimenti resterebbe solo nel buffer locale)
        producer.abortTransaction();  // Annulla la transazione (i dispari sono abortiti)
    }
}
producer.close();

// ========================================================================
// 2. TRANSACTIONAL CONSUMER
// ========================================================================
final boolean readUncommitted = false; // false -> "read_committed", true -> "read_uncommitted"

final Properties consumerProps = new Properties();
consumerProps.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
consumerProps.put(ConsumerConfig.GROUP_ID_CONFIG, "groupA");
if (readUncommitted) {
    // Legge TUTTI i messaggi (sia pari committati sia dispari abortiti)
    consumerProps.put(ConsumerConfig.ISOLATION_LEVEL_CONFIG, "read_uncommitted");
} else {
    // Legge SOLO i messaggi appartenenti a transazioni committate (solo i pari)
    consumerProps.put(ConsumerConfig.ISOLATION_LEVEL_CONFIG, "read_committed");
}
consumerProps.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "latest");
consumerProps.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
consumerProps.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());

KafkaConsumer<String, String> consumer = new KafkaConsumer<>(consumerProps);
consumer.subscribe(Collections.singletonList("topicA"));
```

4.4 Atomic Forwarder Stateless: End-to-End Exactly-Once (EOS) con Gestione Errori e `seek`

(Rif: `AtomicUpperCaseConsumer.java`, `AtomicForwarder.java`, `TransactionalForwarderTemplate.java`)

```
// Se avvii più istanze nello stesso gruppo ("groupA"), passa un transactionalId DIVERSO in args[0] per ciascuna!
String transactionalId = args.length >= 1 ? args[0] : "forwarderTransactionalId";
String groupId = args.length >= 2 ? args[1] : "groupA";

// ========================================================================
// 1. CONSUMER: legge solo messaggi committati e NON fa auto-commit
// ========================================================================
final Properties consumerProps = new Properties();
consumerProps.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
consumerProps.put(ConsumerConfig.GROUP_ID_CONFIG, groupId);
consumerProps.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
consumerProps.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
consumerProps.put(ConsumerConfig.ISOLATION_LEVEL_CONFIG, "read_committed");
// Il consumer NON committa da solo: gli offset vengono salvati dentro la transazione del producer!
consumerProps.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, String.valueOf(false));
consumerProps.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");

KafkaConsumer<String, String> consumer = new KafkaConsumer<>(consumerProps);
consumer.subscribe(Collections.singletonList("topicA"));

// ========================================================================
// 2. PRODUCER: transazionale + idempotente
// ========================================================================
final Properties producerProps = new Properties();
producerProps.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
producerProps.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());
producerProps.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());
producerProps.put(ProducerConfig.TRANSACTIONAL_ID_CONFIG, transactionalId);
producerProps.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, String.valueOf(true));

final KafkaProducer<String, String> producer = new KafkaProducer<>(producerProps);
producer.initTransactions();

// ========================================================================
// 3. CICLO ATOMICO READ-PROCESS-WRITE CON ROLLBACK E SEEK SU ERRORE
// ========================================================================
while (true) {
    final ConsumerRecords<String, String> records = consumer.poll(Duration.of(5, ChronoUnit.MINUTES));
    if (records.isEmpty()) {
        continue;
    }

    try {
        producer.beginTransaction();

        // A) Elabora i record letti e produce i risultati sul topic di destinazione
        for (final ConsumerRecord<String, String> record : records) {
            String transformed = record.value().toLowerCase();
            producer.send(new ProducerRecord<>("topicB", record.key(), transformed));
        }

        // B) Calcola per ogni partizione letta il prossimo offset da leggere (lastOffset + 1)
        final Map<TopicPartition, OffsetAndMetadata> map = new HashMap<>();
        for (final TopicPartition partition : records.partitions()) {
            final List<ConsumerRecord<String, String>> partitionRecords = records.records(partition);
            final long lastOffset = partitionRecords.get(partitionRecords.size() - 1).offset();
            map.put(partition, new OffsetAndMetadata(lastOffset + 1));
        }

        // C) Salva atomicamente sia i messaggi prodotti su topicB sia l'avanzamento degli offset di groupA su topicA
        producer.sendOffsetsToTransaction(map, consumer.groupMetadata());
        producer.commitTransaction();
    } catch (ProducerFencedException | OutOfOrderSequenceException e) {
        // Un'altra istanza con lo stesso transactional.id ha fatto zombie fencing: chiudi tutto
        producer.close();
        consumer.close();
        break;
    } catch (KafkaException e) {
        // Errore transitorio: annulla la transazione e riavvolgi gli offset del consumer per rileggere il batch
        producer.abortTransaction();
        for (final TopicPartition partition : records.partitions()) {
            if (consumer.assignment().contains(partition)) {
                consumer.seek(partition, records.records(partition).get(0).offset());
            }
        }
    }
}
```

4.5 Atomic Fault-Tolerant *Stateful* Forwarder (EOS con Stato Locale e Recovery da Topic — Lab 1 Es. 4)

(Rif: `AtomicFaultTolerantForwarder.java`)

```
// ========================================================================
// 1. CONFIGURAZIONE CONSUMER PRINCIPALE
// ========================================================================
final Properties consumerProps = new Properties();
consumerProps.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
consumerProps.put(ConsumerConfig.GROUP_ID_CONFIG, "groupB");
consumerProps.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
consumerProps.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
consumerProps.put(ConsumerConfig.ISOLATION_LEVEL_CONFIG, "read_committed");
consumerProps.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, String.valueOf(false));

KafkaConsumer<String, String> consumer = new KafkaConsumer<>(consumerProps);

// ========================================================================
// 2. RECOVERY DELLO STATO DA outputTopic ("topicB") ALL'AVVIO
//    Usa assign() + seekToBeginning() per leggere tutte le partizioni da offset 0
// ========================================================================
final Map<String, Integer> countMap = new HashMap<>();
final Properties storeProps = new Properties();
storeProps.putAll(consumerProps);
storeProps.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, IntegerDeserializer.class.getName());

try (KafkaConsumer<String, Integer> storeConsumer = new KafkaConsumer<>(storeProps)) {
    List<PartitionInfo> infos = storeConsumer.partitionsFor("topicB");
    if (infos != null && !infos.isEmpty()) {
        List<TopicPartition> partitions = infos.stream()
                .map(p -> new TopicPartition("topicB", p.partition()))
                .toList();
        storeConsumer.assign(partitions);
        storeConsumer.seekToBeginning(partitions);
        while (true) {
            ConsumerRecords<String, Integer> stored = storeConsumer.poll(Duration.ofSeconds(1));
            if (stored.isEmpty()) break;
            stored.forEach(r -> countMap.put(r.key(), r.value()));
        }
    }
}

// Solo DOPO aver ricostruito countMap, si iscrive a inputTopic ("topicA")
consumer.subscribe(Collections.singletonList("topicA"));

// ========================================================================
// 3. PRODUCER TRANSAZIONALE (Valore di output = Integer per il contatore)
// ========================================================================
final Properties producerProps = new Properties();
producerProps.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
producerProps.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());
producerProps.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, IntegerSerializer.class.getName());
producerProps.put(ProducerConfig.TRANSACTIONAL_ID_CONFIG, "atomicCountForwarderId");
producerProps.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, String.valueOf(true));

final KafkaProducer<String, Integer> producer = new KafkaProducer<>(producerProps);
producer.initTransactions();

// ========================================================================
// 4. CICLO ATOMICO STATEFUL: AGGIORNA CONTATORE + PUBBLICA + COMMITTA OFFSET
// ========================================================================
while (true) {
    final ConsumerRecords<String, String> records = consumer.poll(Duration.of(5, ChronoUnit.MINUTES));
    if (records.isEmpty()) continue;

    producer.beginTransaction();
    for (final ConsumerRecord<String, String> record : records) {
        final int newCount = countMap.compute(record.key(), (k, v) -> v == null ? 1 : v + 1);
        producer.send(new ProducerRecord<>("topicB", record.key(), newCount));
        System.out.println("Key: " + record.key() + "\tCount: " + newCount);
    }

    final Map<TopicPartition, OffsetAndMetadata> map = new HashMap<>();
    for (final TopicPartition partition : records.partitions()) {
        final List<ConsumerRecord<String, String>> partitionRecords = records.records(partition);
        final long lastOffset = partitionRecords.get(partitionRecords.size() - 1).offset();
        map.put(partition, new OffsetAndMetadata(lastOffset + 1));
    }

    producer.sendOffsetsToTransaction(map, consumer.groupMetadata());
    producer.commitTransaction();
}
```

4.6 Creazione Programmatica dei Topic e *Log Compaction* (`AdminClient` / `TopicManager`)

(Rif: `TopicManager.java`)

```
Properties props = new Properties();
props.put(AdminClientConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");

try (AdminClient adminClient = AdminClient.create(props)) {
    // 1. Elenca i topic esistenti ed elimina il topic se già presente
    Set<String> topicsNames = adminClient.listTopics().names().get();
    if (topicsNames.contains(topicName)) {
        adminClient.deleteTopics(Collections.singletonList(topicName)).all().get();
        Thread.sleep(5000); // Attende il completamento della cancellazione sul broker
    }

    // 2. Crea il nuovo topic specificando (nome, numeroPartizioni, replicationFactor)
    NewTopic newTopic = new NewTopic(topicName, topicPartitions, replicationFactor);

    // 3. Opzionale: attiva la Log Compaction (mantiene solo l'ultimo valore per ogni key)
    if (compact) {
        Map<String, String> topicConfigs = new HashMap<>();
        topicConfigs.put(TopicConfig.CLEANUP_POLICY_CONFIG, TopicConfig.CLEANUP_POLICY_COMPACT);
        topicConfigs.put(TopicConfig.MIN_CLEANABLE_DIRTY_RATIO_CONFIG, "0.01");
        topicConfigs.put(TopicConfig.SEGMENT_MS_CONFIG, "5000");
        newTopic.configs(topicConfigs);
    }

    adminClient.createTopics(Collections.singletonList(newTopic)).all().get();
}
```

5. Guida Completa agli Esercizi del Lab 1 (Domande, Risposte e Soluzioni)

(Rif: Slide `kafka_Lab1.pdf` e codice in `kafka/Lab/Lab1`)

Esercizio	Classi nel Progetto	Obiettivo Principale	Concetto Chiave / Risposta alle Domande delle Slide
Exercise 1 *(Due Consumer base sullo stesso topic senza garanzie)*	`BasicProducer.java` `PrintConsumer.java` (C_1) `UpperCaseConsumer.java` (C_2)	C_1 stampa i record di `topicA` ; C_2 converte i valori in minuscolo e li pubblica su `topicB` senza garanzie di consegna (`autoCommit = true` ogni 15s).	- **C_1 e C_2 devono avere `group.id` DIVERSI** (`"groupB"` e `"groupA"`) affinché entrambi ricevano tutti i messaggi di `topicA` (broadcast). - **Cosa succede se un consumer fallisce?**: l'altro consumer continua a funzionare normalmente perché i gruppi sono indipendenti. - **Cosa succede quando lo riavvii?**: riparte dall'**ultimo offset salvato dall'auto-commit** (ignorando `auto.offset.reset`). I messaggi già elaborati dopo l'ultimo auto-commit ma prima del crash vengono **riletti e duplicati** su `topicB` (*At-Least-Once*), oppure se l'auto-commit era scattato prima della fine del ciclo `for` , i messaggi residui del batch vanno **persi** (*At-Most-Once*).
Exercise 2 *(Parallelizzazione di un Consumer lento)*	`UpperCaseConsumer.java` `TopicManager.java`	C_2 è lento (`Thread.sleep(700)` contro i `500ms` del producer) e accumula ritardo (*consumer lag*). Come migliorare le performance?	- **Come migliorare il throughput**: (1) aumentare il numero di partizioni *P* di `topicA` (con `TopicManager`), e (2) avviare più istanze di C_2 con lo **STESSO `group.id = "groupA"`** . - **Cosa succede se avvii più istanze ($C \le P$)?**: Kafka esegue un *rebalance* e divide le *P* partizioni tra le *C* istanze (es. con $P=2$ e 2 istanze, ciascuna legge 1 partizione dimezzando il carico). Se $C > P$, le $C - P$ istanze in eccesso restano **inattive (idle)**. - **Cosa succede se un'istanza fallisce?**: il broker rileva il crash (mancato heartbeat) e riassegna automaticamente le partizioni dell'istanza morta alle istanze superstiti dello stesso gruppo.
Exercise 3 *(Consumer C_2 con semantica Exactly-Once Stateless)*	`AtomicUpperCaseConsumer.java`	Garantire che ogni messaggio di `topicA` venga trasformato e scritto su `topicB` **una e una sola volta (EOS)**.	- **Configurazione EOS**: `enable.auto.commit = false` , `isolation.level = "read_committed"` (sia su C_2 sia su eventuali consumer a valle che leggono `topicB` !), producer con `transactional.id` e `enable.idempotence = true` . - **Ciclo atomico**: `beginTransaction()` → `send(topicB, ...)` → `sendOffsetsToTransaction(lastOffset + 1, groupMetadata)` → `commitTransaction()` . - **Se un'istanza fallisce**: la transazione aperta viene abortita dal broker; i consumer `"read_committed"` su `topicB` ignorano i messaggi parziali e al riavvio (o dopo il rebalance su un'altra istanza) gli offset su `topicA` ripartono dall'ultima transazione committata. - **Se avvii più istanze**: devono avere lo **stesso `group.id`** ma **`transactional.id` UNIVOCI** per ogni istanza, altrimenti scatta `ProducerFencedException` (*zombie fencing*).
Exercise 4 *(Consumer C_2 Stateful: contatore di messaggi per chiave con EOS)*	`AtomicFaultTolerantForwarder.java` `BasicProducer.java`	Modificare C_2 per contare e inoltrare il numero totale di messaggi ricevuti per ciascuna `key` . Valutare fault tolerance su 1 istanza e su più istanze.	- **Singola istanza — Cosa succede in caso di crash se teniamo il contatore solo in RAM?**: al riavvio la `HashMap` è vuota e i contatori ripartono da `1` (NON garantisce EOS!). - **Come garantire EOS su singola istanza**: poiché il nuovo conteggio è salvato atomicamente su `topicB` (configurato con **Log Compaction**), all'avvio usiamo uno `storeConsumer` (`assign` + `seekToBeginning` + `read_committed`) per rileggere `topicB` e ricostruire `countMap` prima di fare `subscribe("topicA")` . - **Cosa cambia con PIÙ istanze?**: grazie al partizionamento per `key` , la stessa chiave va sempre alla stessa partizione e quindi alla stessa

Esercizio	Classi nel Progetto	Obiettivo Principale	Concetto Chiave / Risposta alle Domande delle Slide
			istanza di C_2. Tuttavia, se un'istanza cade o entra a caldo causando un **rebalance**, l'istanza che eredita la partizione non ha in RAM gli ultimi conteggi prodotti dall'istanza precedente: serve un `ConsumerRebalanceListener` che rilegga `topicB` dentro `onPartitionsAssigned(...)` a ogni rebalance!